

Full Stack Developer — Step 3

React

Roadmap objective: Build component-based interactive user interfaces

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Components
- JSX
- Props
- State
- Events
- Lists/keys
- Effects
- Forms
- Routing concept
- API integration

Core concepts

1. React encourages UI composition from reusable components.
2. Props pass data into components; state represents data that can change and affect rendering.
3. Effects are for synchronizing with external systems; do not use them for every piece of derived data.
4. Stable keys help React identify list items during reconciliation.

Concept map

```
React
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
+--> Document the decision
```

Detailed study guide

1. Components

Study components through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. JSX

Study jsx through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. Props

Study props through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. State

Study state through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Events

Study events through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. Lists/keys

Study lists/keys through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

React component

```
function CareerCard({ career }) {
  return (
    <article>
      <h2>{career.title}</h2>
      <p>{career.description}</p>
      <button onClick={() => console.log(career.id)}>
        View roadmap
      </button>
    </article>
  );
}
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Props	Input data	Parent to child
State	Changing component data	Drives rendering
Effect	External synchronization	Use deliberately

Common mistakes

- Huge components
- Using effects for derived values
- Unstable list keys
- Prop drilling without considering component design
- Fetching data without cleanup/error handling

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Create a reusable component.
- Write down what you expected, what happened, and why.

Lab 2

- Pass data using props.
- Write down what you expected, what happened, and why.

Lab 3

- Add local state.
- Write down what you expected, what happened, and why.

Lab 4

- Render a list with stable keys.
- Write down what you expected, what happened, and why.

Lab 5

- Add a form event.
- Write down what you expected, what happened, and why.

Lab 6

- Integrate one API request and decide whether an effect is actually needed.
- Write down what you expected, what happened, and why.

Mini-project

Build a React roadmap viewer with reusable cards, props, state, forms and API integration. Keep components small and give list items stable keys.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

Props vs state?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why do lists need keys?

- Answer structure: definition → example → trade-off/limitation → practical use.

When should an effect be used?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do events work?

- Answer structure: definition → example → trade-off/limitation → practical use.

How would you split a large component?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you fetch API data safely?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Component	Write this in your own words.	Give one project example.
JSX	Write this in your own words.	Give one project example.
Props	Write this in your own words.	Give one project example.
State	Write this in your own words.	Give one project example.
Effect	Write this in your own words.	Give one project example.
Render	Write this in your own words.	Give one project example.
Key	Write this in your own words.	Give one project example.
Event handler	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Hook	Write this in your own words.	Give one project example.
Reconciliation	Write this in your own words.	Give one project example.

Self-test

- Explain Components without using its textbook definition.
- Show a small example of JSX.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Components.
- I can explain and demonstrate JSX.
- I can explain and demonstrate Props.
- I can explain and demonstrate State.
- I can explain and demonstrate Events.
- I can explain and demonstrate Lists/keys.
- I can explain and demonstrate Effects.
- I can explain and demonstrate Forms.
- I can explain and demonstrate Routing concept.
- I can explain and demonstrate API integration.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://react.dev/>

Recommended videos

- React Tutorial (Existing DB resource 99)
- CodeWithHarry React playlist (Existing DB resource 230)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.